

explain me csv module

Python csv Module

The Python `csv` module is a built-in module used to **read and write CSV (Comma-Separated Values) files**.

For your Data Engineering path, it is useful because CSV files are very common as **input/output data sources** before processing them with **PySpark, Pandas, databases, or ETL pipelines**.

1. Import the module

```
</> Python
```

[Run](#)

```
import csv
```

No installation is required because `csv` is part of Python's standard library.

2. What is a CSV file?

A CSV file stores tabular data as text.

Example: `employees.csv`

```
</> CSV
```



```
id,name,department,salary
1,Ahmed,IT,8000
2,Omar,HR,7000
3,Youssef,Data,10000
```

Conceptually:

id	name	department	salary
1	Ahmed	IT	8000

id	name	department	salary
2	Omar	HR	7000
3	Youssef	Data	10000

The comma separates the columns.

3. Reading a CSV file

The basic pattern is:

</> Python



▶ Run

```
import csv

with open("employees.csv", "r") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Output:

```
['id', 'name', 'department', 'salary']
['1', 'Ahmed', 'IT', '8000']
['2', 'Omar', 'HR', '7000']
['3', 'Youssef', 'Data', '10000']
```



Important

`csv.reader()` returns each row as a **list of strings**.

Even:

```
8000
```



is returned as:

</> Python



▶ Run

```
"8000"
```

not:

```
</> Python
```

[Run](#)

```
8000
```

You can convert it:

```
</> Python
```

[Run](#)

```
salary = int(row[3])
```

4. Why `newline=""` ?

When working with CSV files, the recommended pattern is:

```
</> Python
```

[Run](#)

```
with open("employees.csv", "r", newline="") as file:
    reader = csv.reader(file)

    for row in reader:
        print(row)
```

For writing:

```
</> Python
```

[Run](#)

```
with open("employees.csv", "w", newline="") as file:
    writer = csv.writer(file)
```

`newline=""` helps Python's CSV module handle line endings correctly, especially across operating systems.

5. Skip the header

Suppose:

```
</> CSV
```



```
id,name,department,salary
1,Ahmed,IT,8000
2,Omar,HR,7000
```

You can use:

</> Python



▶ Run

```
import csv

with open("employees.csv", "r", newline="") as file:
    reader = csv.reader(file)

    next(reader) # skip header

    for row in reader:
        print(row)
```

Output:

```
['1', 'Ahmed', 'IT', '8000']
['2', 'Omar', 'HR', '7000']
```



6. Access individual columns

</> Python



▶ Run

```
import csv

with open("employees.csv", "r", newline="") as file:
    reader = csv.reader(file)

    next(reader)

    for row in reader:
        employee_id = row[0]
        name = row[1]
        department = row[2]
        salary = int(row[3])

        print(name, department, salary)
```

Output:

```
Ahmed IT 8000
Omar HR 7000
Youssef Data 10000
```



7. csv.DictReader

This is **very useful**.

Instead of accessing:

```
</> Python
```

```
row[0]  
row[1]  
row[2]
```



▶ Run

you can use column names.

```
</> Python
```

```
import csv
```

```
with open("employees.csv", "r", newline="") as file:  
    reader = csv.DictReader(file)
```

```
    for row in reader:  
        print(row)
```



▶ Run

Output looks like:

```
</> Python
```

```
{  
    'id': '1',  
    'name': 'Ahmed',  
    'department': 'IT',  
    'salary': '8000'  
}
```



▶ Run

Now you can do:

```
</> Python
```

```
for row in reader:  
    print(row["name"])  
    print(row["department"])  
    print(row["salary"])
```



▶ Run

This is much easier to read.

8. Convert values

Remember that CSV values are strings.

</> Python



▶ Run

```
import csv

with open("employees.csv", "r", newline="") as file:
    reader = csv.DictReader(file)

    for row in reader:
        employee_id = int(row["id"])
        salary = int(row["salary"])

        print(employee_id, salary)
```

You can also convert:

</> Python



▶ Run

```
age = int(row["age"])
```

</> Python



▶ Run

```
salary = float(row["salary"])
```

9. Writing a CSV file

Use `csv.writer()`.

</> Python



▶ Run

```
import csv

with open("employees.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["id", "name", "department", "salary"])

    writer.writerow([1, "Ahmed", "IT", 8000])
    writer.writerow([2, "Omar", "HR", 7000])
    writer.writerow([3, "Youssef", "Data", 10000])
```

This creates:

</> CSV



```
id,name,department,salary
1,Ahmed,IT,8000
2,Omar,HR,7000
3,Youssef,Data,10000
```

10. Write multiple rows

Use `writerows()` :

</> Python



Run

```
import csv

employees = [
    [1, "Ahmed", "IT", 8000],
    [2, "Omar", "HR", 7000],
    [3, "Youssef", "Data", 10000]
]

with open("employees.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["id", "name", "department", "salary"])
    writer.writerows(employees)
```

11. DictWriter

Similar to `DictReader` , but for writing.

</> Python



Run

```
import csv

employees = [
    {
        "id": 1,
        "name": "Ahmed",
        "department": "IT",
        "salary": 8000
    },
    {
        "id": 2,
        "name": "Omar",
        "department": "HR",
        "salary": 7000
    }
]

with open("employees.csv", "w", newline="") as file:

    fieldnames = ["id", "name", "department", "salary"]

    writer = csv.DictWriter(
        file,
        fieldnames=fieldnames
    )

    writer.writeheader()
    writer.writerows(employees)
```

Result:

</> CSV



```
id,name,department,salary
1,Ahmed,IT,8000
2,Omar,HR,7000
```

12. Append data to an existing CSV

If you want to add rows without deleting existing data:

</> Python



▶ Run


```
import csv

with open("employees.csv", "a", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([4, "Mohamed", "Finance", 9000])
```

"a" means **append**.

The main modes you'll use are:

Mode	Meaning
"r"	Read
"w"	Write / overwrite
"a"	Append

13. Different delimiters

CSV doesn't necessarily have to use commas.

For example:

```
id;name;salary
1;Ahmed;8000
2;Omar;7000
```



You can specify the delimiter:

```
</> Python

import csv

with open("employees.csv", "r", newline="") as file:
    reader = csv.reader(file, delimiter=";")

    for row in reader:
        print(row)
```



▶ Run

You can also write with ; :

</> Python



Run

```
writer = csv.writer(file, delimiter=";")
```

14. Quoted values

CSV handles values containing commas.

Example:

</> CSV



```
id,name,address  
1,Ahmed,"Cairo, Egypt"  
2,Omar,"Giza, Egypt"
```

Using:

</> Python



Run

```
import csv  
  
with open("employees.csv", "r", newline="") as file:  
    reader = csv.DictReader(file)  
  
    for row in reader:  
        print(row["address"])
```

Output:

```
Cairo, Egypt  
Giza, Egypt
```



The CSV module handles the quotes for you.

15. Different CSV dialects

Python also supports CSV dialects.

For example:

</> Python



Run

```
csv.reader(  
    file,  
    delimiter=","  
    quotechar='''  
)
```

Common options include:

</> Python



Run

```
delimiter=","  
quotechar='''  
escapechar="\\\"
```

You normally don't need to change these for ordinary CSV files.

16. Complete practical example

Let's create a CSV containing sales:

</> Python



Run

```
import csv  
  
sales = [  
    [1, "Laptop", 2, 1000],  
    [2, "Mouse", 5, 20],  
    [3, "Keyboard", 3, 50],  
    [4, "Monitor", 2, 300]  
]  
  
with open("sales.csv", "w", newline="") as file:  
  
    writer = csv.writer(file)  
  
    writer.writerow([  
        "id",  
        "product",  
        "quantity",  
        "price"  
    ])  
  
    writer.writerows(sales)
```

Now read it:

</> Python



Run

```
import csv

with open("sales.csv", "r", newline="") as file:

    reader = csv.DictReader(file)

    for row in reader:

        quantity = int(row["quantity"])
        price = float(row["price"])

        total = quantity * price

    print(
        row["product"],
        total
    )
```

Output:

```
Laptop 2000.0
Mouse 100.0
Keyboard 150.0
Monitor 600.0
```



This is actually a small **ETL-style operation**:

```
CSV
↓
Read
↓
Convert data types
↓
Transform
↓
Calculate total
↓
Output
```



17. CSV → JSON

Since you're learning Python for streaming, this is particularly useful.

You can read CSV and create JSON:

</> Python



▶ Run

```
import csv
import json

data = []

with open("employees.csv", "r", newline="") as file:

    reader = csv.DictReader(file)

    for row in reader:
        row["id"] = int(row["id"])
        row["salary"] = int(row["salary"])

        data.append(row)

with open("employees.json", "w") as file:
    json.dump(data, file, indent=4)
```

CSV:

</> CSV



```
id,name,department,salary
1,Ahmed,IT,8000
2,Omar,HR,7000
```

JSON:

</> JSON



```
[
  {
    "id": 1,
    "name": "Ahmed",
    "department": "IT",
    "salary": 8000
  },
  {
    "id": 2,
    "name": "Omar",
    "department": "HR",
    "salary": 7000
  }
]
```

This is useful for your **Python → JSON → Spark Structured Streaming** practice.

18. csv vs Pandas

You should understand the difference.

CSV

```
</> Python  
import csv

with open("employees.csv") as file:
    reader = csv.DictReader(file)

    for row in reader:
        print(row["name"])
```

Good for:

- Simple CSV reading/writing
- Python scripts
- Data generation
- ETL scripts
- Lightweight processing
- Streaming-data generators

Pandas

```
</> Python  
import pandas as pd

df = pd.read_csv("employees.csv")
```

Good for:

- Data analysis
- Filtering
- Grouping
- Aggregation
- Missing values
- Data cleaning
- DataFrames

For your career, **learn both**, but you don't need to memorize every `csv` feature.

19. What you should learn from `csv`

For your Data Engineering/Streaming path, focus on these:

```
csv
├── csv.reader()
├── csv.writer()
├── csv.DictReader()
├── csv.DictWriter()
├── writerow()
├── writerows()
├── read/write/append modes
│   ├── "r"
│   ├── "w"
│   └── "a"
├── delimiter
├── newline=""
└── CSV → JSON
```

The **most important** ones are:

`</>` Python

```
csv.reader()
csv.writer()

csv.DictReader()
csv.DictWriter()

writerow()
writerows()
```

For the kind of streaming practice you're doing, I'd especially recommend mastering `DictReader` + `DictWriter` + **CSV → JSON conversion**, because they make it easy to generate realistic event data that you can then feed into **Spark Structured Streaming** or **Kafka**.



ChatGPT can make mistakes. Check important info.